kiutra API

kiutra

Dec 16, 2024

API

1 API Client **5**

2 API Manager **7**

3 Controller **9**
- 3.1 Cryostat Control . . . 10
- 3.2 Sample Control . . . 13
- 3.3 Temperature Control . . . 16
- 3.4 ADR Control . . . 21
- 3.5 Heater Control . . . 25
- 3.6 Magnet Control . . . 28
- 3.7 Utility Control . . . 32

4 Devices **35**
- 4.1 Heatswitch . . . 35
- 4.2 Pressure Regulator . . . 36
- 4.3 Thermometer . . . 37
- 4.4 Magnet Power Supply . . . 40
- 4.5 Baffles . . . 43
- 4.6 Valve . . . 44
- 4.7 Winch . . . 45
- 4.8 Warmup Heater . . . 46
- 4.9 Gauge . . . 48
- 4.10 Compressor . . . 49
- 4.11 Pumps . . . 50

Index **55**

The kiutra API offers easy access to our cryostat's control operations. The API is implemented as an RPC-server (remote procedure call). Using a device name (like *controller_1*) in conjunction with a procedure key (like *procedure_1*), values can be read, set and procedures executed. Controller objects and relevant hardware components are represented as software instances as illustrated in the image below. Each controller/device implements a set of remote procedures which are accessible over the RPC-server.

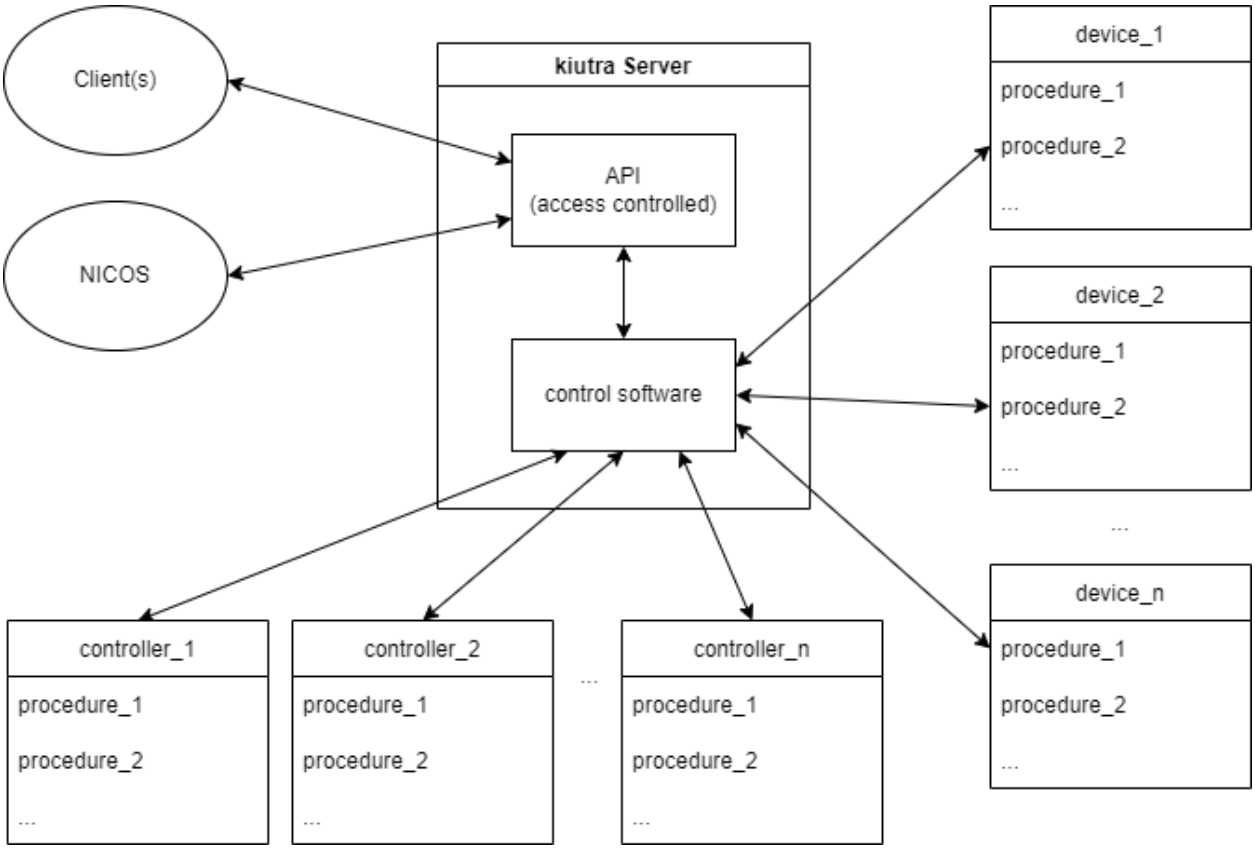

Fig. 1: Clients communicate via a RPC-Server-based API with the kiutra control software and associated peripherals. Each controller/device implements API-accessible procedures.

Note

Multiple clients can interact simultaneously with the server.

Using the provided kiutra clients, temperature control tasks, like sweeps, can be implemented in a few lines of code:

```
# create temperature control client object
temperature_control = TemperatureControl('temperature_control', '192.168.XX.XX')
# start a temperature control with a setpoin of 500 mK/0.5 K and a ramp of 0.15 K/min
temperature_control.start((0.5, 0.15))
# get status code and message
code, message = temperature_control.status
# get actual temperature in K
```

(continues on next page)

(continued from previous page)

```
temp = temperature_control.kelvin
# check whether control process is active
control_active = temperature_control.is_active
# check whether setpoint temperature is reached and stable
temp_stable = temperature_control.is_stable
```

Note

Certain operations require extended access rights to avoid unintentional changes to the system, which could negatively influence performance.

To temporarily unlock a higher access level for lower level operations, simply call the *unlock* procedure of the api manager providing password and access level.

```
# create api manager client object
api_manager = APIManager('192.168.XX.XX')
# unlock admin level access (returns bool indicating success)
success = api_manager.unlock('password', 'admin')
# check access level
level = api_manager.access_level
# lock access
api_manager.lock()
# check access level
level = api_manager.access_level
```

To get a list of accessible controller and device objects, simply run the following piece of code:

```
api_manager = APIManager('192.168.XX.XX')
devices = api_manager.api_info
```

For additional information about a controller or device object, you can run the following command:

```
api_info_dict = <device_name>.api
```

This will yield a dictionary containing further dictionaries with information about the respective function or property. For the *api_manager* it would look something like this:

```
{'unlock': {'permissions': {'user': 'execute', 'admin': 'execute'},
            'signature': '<Signature (hash: str, level: str) -> bool>',
            'doc': '\n Unlock admin access.\n\n Args:\n hash: Passphrase hash using␣
↪werkzeug.security sha1\n level: level like e.-g. \'admin\'\n\n'},
 'lock': {'permissions': {'user': 'execute', 'admin': 'execute'},
          'signature': '<Signature ()>',
          'doc': '\n Immediately locks API access to base level\n\n '},
 'access_level': {'permissions': {'user': 'read', 'admin': 'read'},
                  'input_doc': None,
                  'output': 'str',
                  'output_doc': '\n Access level as string.\n return:\n '},
 'api_info': {'permissions': {'user': 'read', 'admin': 'read'},
              'input_doc': None,
              'output_doc': '\n List of available api devices\n\n Returns (list):␣
↪available api devices as string handles\n\n '},
 'status': {'permissions': {'user': 'read', 'admin': 'read'},
```

(continues on next page)

(continued from previous page)

```
            'input_doc': None,
            'output_doc': '\n Return a tuple of int,string with status information.\n\
↪n Returns (int, str): (DeviceStatus.value, status message)\n\n'}
 }
```

Attention

The above is a preliminary representation and might change.

This package offers a basic, ready to use python client, as well as controller/device specific clients that separately implement all necessary, remote procedure calls. The documentation states each API handle used in the respective method.

We recommend using the jsonrpclib-pelix pypi package. Other implementations might be used, but were not tested.

CHAPTER 1

API Client

api_client

Note

This describes a working state under active development.

Kiutra devices host a JSON-RPC server. By sending remote procedure calls using the JSON-RPC protocol, kiutra devices can be interfaced from different programming languages. Messages to the server must be formatted in the form of *device*.**key**, where **key** can be one of the documented keywords and *device* is the device instance identifier respectively. The keywords are the names of the respective client implementation. To query a value use the remote procedure **read**, to set a value use **write** and to execute a procedure use **call**. Available commands can be found in the code documentation by clicking on [source] of the according section. Device name plus property/method name yield the api handle.

A basic client object to control the temperature may be created by simply following the code example given in the documentation of *KiutraClient*.

class api_client.**KiutraClient**(*host*, *port=1006*, *max_retry=10*)

A class that provides an interface to the kiutra RPC server. The RPC server usually implements the interface to all hardware and guarantees serialization of device communication. Use the basic client as follows:

```
from kiutra_api.api_client import KiutraClient
if __name__=='__main__':
    client = KiutraClient('192.168.XX.XX')
    # Get controller temperature
    client.query('temperature_control.kelvin')
    # Ramp to 50 K with a ramp rate of 0.25 K/min
    client.call('temperature_control.start', (50, 0.25))
    # Start a sequence first stabilizing 0.1 K for 120 s with a ramp rate of
    # 0.15 K/min, subsequently ramping to 5 K with a ramp rate of 0.2 K/min.
```

(continues on next page)

(continued from previous page)

```
    client.call('temperature_control.start_sequence',
                [(0.1, 0.15, 120), (5.0, 0.2, 0)])
```

Alternatively, the pre defined client classes provided in this package are ready to use, offering all necessary interfacing options to give you full control of your cryostat.

call(*key*, **args*, ***kwargs*)

Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

query(*key*, **args*)

Queries a parameter based on key.

Parameters

key – parameter handle like device.parameter

set(*key*, *value*)

Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

CHAPTER 2

API Manager

device name: api_manager

The API manager offers access to information about the available controller and device objects based on the hardware configuration. It also enables unlocking of admin-level access.

class api_client.**APIManager**(*host*, *port=1006*, *max_retry=10*)

Bases: Device

The API manager devices handles unlocking and locking of protected access levels. It also provides an overview of available api accessible devices.

property access_level: str

Access level as string.

property api: dict

API documentation computed at runtime based on the device implementation.

property api_info

List of available api devices

Returns (list): available api devices as string handles

call(*key*, **args*, ***kwargs*)

Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

lock()

Locks API access to base level

query(*key*, **args*)

Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

reset()
Resets device.

restart_service()
Restarts kiutra software. This will stop any ongoing control process und close all valves.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

unlock(*pw: str*, *level: str*) → bool
Unlock protected access level for 5 minutes.

Parameters

- **pw** – Passphrase
- **level** – level like e.-g. ‘admin’

unlock_with_timeout(*pw: str*, *level: str*, *timeout: float = 5*) → bool
Unlock protected access level for timeout minutes. This method is not compatible with older cryostat software.

Parameters

- **pw** – Passphrase
- **level** – level like e.-g. ‘admin’
- **timeout** – timeout after which api-rstrictions will be re-applied

CHAPTER 3

Controller

The kiutra API offers access to all main control objets. These controllers are responsible for higher-level procedures like temperature control, gashandling and sample transfers. Below is an overview of available controllers and their device handles:

Table 1: Controller Objects

Interface	Device(s)
Temperature Control	**temperature_control**: Full-range continuous temperature control object combining both resistive heating and magnetic cooling/heating.
ADR Control	**adr_control, adr_control_1, adr_control_2**: ADR control object to control the main ADR system (*adr_control*) or individual adr units (*adr_control_1, adr_control_2*).
Heater Control	**sample_heater**: Control object to control the resistive sample stage heater.
Magnet Control	**sample_magnet**: Control object to control superconducting sample magnet(s).
Sample Control	**sample_changer**: Control object to control sample transfers.
Cryostat Control	**cryostat/cryostat_control**: Control object to execute cryostat-wide operations like cooling down, warming up and purging the system.
Utility Control	**utility_control**: Control object to execute utility operations in case of manual control like closing or opening valves.

3.1 Cryostat Control

device name: cryostat_control

The cryostat control object allows you to execute system-wide operations like evacuating the cryostat, cooling it down, warming it up, or venting the dewar. It also collects and analyzes metrics, indicating the overall state of the system. These values include the status of the pre-cooling unit, its temperature, and the dewar pressure.

class controller_interfaces.**CryostatControl**(*device*, *host*, *port=1006*, *max_retry=10*)

Bases: *Control*

A *CryostatControl* object offers the possibility to control operations concerning the entire cryostat like evacuating the dewar or warming up the system. It also offers access to system-wide device metrics and monitoring information.

Use client device like illustrated below:

```
cryostat_control = CryostatControl('cryostat', '192.168.XX.XX')
code, message = cryostat_control.status
cryostat_control.cooldown()
```

To start a cooldown from an arbitrary RPC client, use **cryostat_control.cooldown** and the remote procedure **call**.

Attention

Some options might not be available depending on the hardware configuration.

property api: dict

API documentation computed at runtime based on the device implementation.

property blocking_devices: list

Gets list of device names blocking the use of the controller.

call(*key*, **args*, ***kwargs*)

Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property check_idle_state: bool

Gets indicator whether sequence is in idle state and ready to use.

check_magnets(***kwargs*)

Waits for magnets to be empty. Stops after timeout.

clear_magnets(***kwargs*)

Initiates discharging of all magnets.

close_baffles(***kwargs*)

Safely close the baffles.

close_gate(**kwargs*)
Safely close the gate.

cooldown(**kwargs*)
Cools down the cryostat

cooldown_block(**kwargs*)
Safely switch on cryocooler and monitor cooldown.

cooldown_only(**kwargs*)
Cooldown system after it has been evacuated.

property detailed_progress: float
Gets detailed progress of controller sequence.

device_check()
Runs check whether all devices are ready to use.

property display_progress: float
Gets display progress of sequence in %.

evacuate(**kwargs*)
Evacuates the cryostat

initialize(**kwargs*)
Initializes system components. This will e.,g., home the sample loader, set the system air-pressure and check/reset other critical components.

property internal_status_name: str
Gets string representation of internal status name.

property is_active: bool
Gets indicator whether controller is active.

property is_blocked: bool
Gets indicator whether device is blocked.

property is_blocked_by: str
Gets name(s) of blocking device(s) as string.

property kelvin: float
Gets filtered temperature of system thermometer in K.

property name: str
Gets device name.

open_baffles(**kwargs*)
Safely open the baffles.

open_gate(**kwargs*)
Safely open the gate.

property pressure: float
Gets filtered isolation vacuum pressure.

property progress: float
Gets progress of sequence.

pump_airlock(**kwargs)
Evacuate the airlock chamber.

pump_suspend(**kwargs)
Safely stop pumps.

purge(**kwargs)
Purges or vents the cryostat depending on the attached gas.

query(*key*, *args)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

reset()
Resets device.

restart_service()
Restarts kiutra software. This will stop any ongoing control process und close all monostable valves.

return_to_idle(**kwargs)
Returns to a safe state.

property runstatus: bool
Gets cryocooler run status.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

start(*target: str = 'default'*)
Starts control sequence.

start_cryocooler(**kwargs)
Safely switch on cryocooler and monitor cooldown.

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

property step: str
Gets string representation of control sequence step.

stop()
Stops execution of current control sequence.

stop_cryocooler(**kwargs)
Stops the cold head

stop_heater(**kwargs)
Stops any heating process

stop_pumps(**kwargs)
Stops the pumps

property successful_termination: bool
Gets indicator whether last controller process terminated successfully.

property temperature_rate: float
Gets linearized temperature change rate of system in K/min.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

trouble_shoot(**kwargs*)
A trouble shoot will be executed to solve communication issues.

vent(**kwargs*)
Warmup and vent system.

vent_airlock(**kwargs*)
Vent the airlock chamber.

vent_dewar(**kwargs*)
Vents the cryostat when system is warm but evacuated.

warmup(**kwargs*)
Stops cold head and actively warms up system if a heater is installed

3.2 Sample Control

device name: sample_control

The sample control object controls sample transfers and monitors involved components. It features a reset of the sample changer in case of an interrupted transfer attempt.

class controller_interfaces.**SampleControl**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: *Control*

The sample control object offers the possibility to control operations concerning the sample transfer.

Use client device like illustrated below:

```
sample_loader = SampleControl('sample_loader', '192.168.XX.XX')
code, message = sample_loader.status
sample_loader.load_sample()
# sample_loader.unload_sample()
```

To start a sample loading process from an arbitrary RPC client, use **sample_loader.load_sample** and the remote procedure **call**.

Attention

Some options might not be available, depending on the hardware configuration.

property api: dict
API documentation computed at runtime based on the device implementation.

property blocking_devices: list
Gets list of device names blocking the use of the controller.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property check_idle_state: bool
Gets indicator whether sequence is in idle state and ready to use.

close_airlock()
Evacuates and seals the airlock.

property detailed_progress: float
Gets detailed progress of controller sequence.

property display_progress: float
Gets display progress of sequence in %.

property door_open: bool
Gets indicator whether the door is open.

property internal_status_name: str
Gets string representation of internal status name.

property is_active: bool
Gets indicator whether controller is active.

property is_blocked: bool
Gets indicator whether device is blocked.

property is_blocked_by: str
Gets name(s) of blocking device(s) as string.

load_sample()
Starts sample loading process. This will require you to open the door at the beginning of the procedure.

property name: str
Gets device name.

open_airlock()
Vents the airlock to be safely opened.

property progress: float
Gets progress of sequence.

pump_airlock()
Evacuates and seals the airlock.

purge_airlock()

Purges the airlock to ambient pressure to speedup warming up the sample using connected purge gas.

query(*key*, **args*)

Queries a parameter based on key.

Parameters

key – parameter handle like device.parameter

reset()

Resets device.

return_to_idle()

Brings the sample loader to a state from which you can safely load/unload a sample.

property rollback_necessary: bool

Gets indicator whether a rollback is necessary (at least one device not ready).

property sample_loaded: bool

Gets (and sets) indicator whether a sample is loaded.

Attention

If your device does not have automatic sample detection, you can override the sample state assumed by the system.

set(*key*, *value*)

Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

short_purge_airlock()

Purges the airlock for a short time to speedup warming up the sample using connected purge gas.

start(*target: str = 'default'*)

Starts control sequence.

property status: (<class 'int'>, <class 'str'>)

Tuple representing the device status as int and a message.

property step: str

Gets string representation of control sequence step.

stop()

Stops execution of current control sequence.

property successful_termination: bool

Gets indicator whether last controller process terminated successfully.

time() → float

Returns current system time as unix timestamp.

Returns: unix time

transfer_sample()

Starts loading process. This will require you to open the door at the beginning of the procedure. This can be used for both loading and unloading but might include unnecessary steps.

unload_sample()

Starts sample unloading process.

vent_airlock()

Vents the airlock to be safely opened.

3.3 Temperature Control

Attention

The universal temperature control object may already be used; however, it is still under active development.

The temperature control object features a hybrid temperature control approach enabling continuous temperature ramps from sub-Kelvin to to room temperatures.

device name: temperature_control

class controller_interfaces.**TemperatureControl**(*device*, *host*, *port=1006*, *max_retry=10*)

Bases: TemperatureSetpointControl

Continuous temperature control object for full-range temperature setpoint and ramp control.

Use client device like illustrated below:

```
temperature_control = TemperatureControl('temperature_control', '192.168.XX.XX')
code, message = temperature_control.status
temperature_control.reset()
temperature_control.start_sequence([(0.1, 0.15, 120), (5.0, 0.2, 0)])
```

property adr_pid: (<class 'float'>, <class 'float'>, <class 'float'>)

Gets adr pid values.

property adr_pid_table: list

Gets specified adr pid table.

property api: dict

API documentation computed at runtime based on the device implementation.

property blocking_devices: list

Gets list of device names blocking the use of the controller.

call(*key*, **args*, ***kwargs*)

Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property check_idle_state: bool

Gets indicator whether sequence is in idle state and ready to use.

property condition: str

Gets string representation of current controller condition.

cooldown()

Cooldown to bath temperature if temperature is above bath temperature.

property detailed_progress: float

Gets detailed progress of controller sequence.

property display_progress: float

Gets display progress of sequence in %.

get_buffer_after(*key*, *t_after=0*) → list

Returns buffer after t_after (unix timestamp als number). :param key: Property name like 'kelvin' or 'sensorunit' :param t_after: timstamp after which buffered values will be returned

Returns: buffered values as 2D matrix.

get_ramping_info()

Returns dictionary containing information about any ongoing ramp or temperature control process.

property heater_pid

Gets heater pid values.

property heater_pid_table: list

Gets specified heater pid table.

property internal_setpoint: float

Gets target temperature curve.

property internal_status_name: str

Gets string representation of internal status name.

property is_active: bool

Gets indicator whether controller is active.

property is_blocked: bool

Gets indicator whether device is blocked.

property is_blocked_by: str

Gets name(s) of blocking device(s) as string.

property kelvin: float

Gets measured temperature of temperature control thermometer in K.

property name: str
Gets device name.

property progress: float
Gets progress of sequence.

propose_control_mode(*setpoint: float*, *ramp: float*, *mode: str = 'ramp'*, *start_temperature: float | None = None*)
Returns a proposed control mode based on the given parameters.

Parameters

- **setpoint** – setpoint temperature to control [K]
- **ramp** – temperature ramp to control [K/min]
- **mode** – Prioritization mode (ramp/stabilize).
- **start_temperature**

Returns:

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

property ramp: float
Gets and sets setpoint ramp.

property ramping: bool
Gets indicator whether controller is ramping.

recharge()
Start adr recharging.

reset()
Resets device.

return_to_idle(**args*, ***kwargs*)
Resets or activates components of the controller to solve potential issues.

Parameters

- ***args**
- ****kwargs**

Returns:

property sections: list
[(float, float, float)].

Type
Gets and sets setpoint sequence sections as list of triples

property sequential: bool
Gets and sets bool indicating whether setpoint sequence should be used.

set(*key*, *value*)

Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property setpoint: float

Gets and sets setpoint.

property stable: bool

Gets indicator whether controller is stable.

start(*target: (<class 'float'>, <class 'float'>) = None*, *setpoint: float = None*, *ramp: float = None*)

Starts control sequence.

Parameters

- **target** – legacy notation for (setpoint, ramp)
- **setpoint** – Final target temperature
- **ramp** – Measurement ramp speed

start_cadr(*setpoint: float*, *ramp: float*, *start_temperature: float | None = None*, *pre_regenerate: bool = True*, *setup_speed: float | None = None*, *init_hold: float = 0.0*)

Start countinuous adr operation.

start_control(***kwargs*)

Generic start method. This is recommended for experienced users and targeted at automations.

Parameters

- ***args**
- ****kwargs**

Returns:

start_heater_control(*setpoint: float*, *ramp: float*, *start_temperature: float | None = None*, *setup_speed: float | None = None*, *init_hold: float = 0*)

Heater only control aplicable above bath temperature.

Parameters

- **setpoint** – setpoint temperature to control [K]
- **ramp** – temperature ramp to control [K/min]
- **start_temperature**
- **setup_speed**
- **init_hold**

Returns:

start_proposed_mode(*setpoint: float*, *ramp: float*, *mode: str = 'ramp'*, *start_temperature: float | None = None*)

Returns the proposed control mode based on the given parameters.

Parameters

- **setpoint** – setpoint temperature to control [K]
- **ramp** – temperature ramp to control [K/min]
- **mode** – Prioritization mode (ramp/stabilize).
- **start_temperature**

Returns:

start_sequence(*sections: list*)

Starts temperature control of setpoint sequence.

sections:
List of triples containing: temperature setpoint, controlled ramp and hold-time: *[(setpoint, ramp, holdtime)]/[(float, float, float)]*

start_single_adr(*setpoint: float*, *ramp: float*, *start_temperature: float | None = None*, *setup_speed: float | None = None*, *init_hold: float = 0.0*, *pre_regenerate: bool = True*)

Smooth single shot using only last adr-stage for seamless operation.

Parameters

- **setpoint** – temperature setpoint in K
- **ramp** – target temperature ramp in K/min
- **start_temperature** – start temperature in K (optional)
- **setup_speed** – ramp in K/min to approach start temperature if specified (optional)
- **init_hold** – time to wait at start temperature in s (optional)
- **pre_regenerate** – regenerate befor starting?

Returns:

property state: str

Gets string representation of the controlling state-machine's current state.

property status: (<class 'int'>, <class 'str'>)

Tuple representing the device status as int and a message.

property step: str

Gets string representation of control sequence step.

stop()

Stops execution of current control sequence.

property successful_termination: bool

Gets indicator whether last controller process terminated successfully.

time() → float

Returns current system time as unix timestamp.

Returns: unix time

wait_done()
Blocks until the controller is not active anymore.

Returns:

wait_stable()
Blocks until the controller is stable. If the controller is inactive, an exception will be raised.

Returns:

3.4 ADR Control

device names: adr_control, adr_control_1, adr_control_2, ...

There are different ADR control objects available. The main control object can be accessed using "adr_control" as device name. Individual ADR units can be accessed by appending a numeric id like "adr_control_id". These units might offer limited functionality.

class controller_interfaces.**ADRControl**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: TemperatureSetpointControl

ADR control object for low temperature setpoint and ramp control.

Use client device like illustrated below:

```
# initialize adr control client object with device object identifier and kiutra␣
↪base IP
adr_control = ADRControl('adr_control', '192.168.XX.XX')
# read control object status
code, message = adr_control.status
# reset adr control object
adr_control.reset()
# start adr temperature control
adr_control.start_adr(setpoint=0.5, ramp=0.1, pre_regenerate=True)
# read current temperature
T = adr_control.kelvin
# update setpoint
adr_control.setpoint = 1.0
```

property adr_mode: int
Gets and sets integer describing the depth of adr operation (soon obsolete).

property api: dict
API documentation computed at runtime based on the device implementation.

property blocking_devices: list
Gets list of device names blocking the use of the controller.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property charge: float
Gets adr charge f(T, B) in %.

property check_idle_state: bool
Gets indicator whether sequence is in idle state and ready to use.

property condition: str
Gets string representation of current controller condition.

property detailed_progress: float
Gets detailed progress of controller sequence.

property display_progress: float
Gets display progress of sequence in %.

get_buffer_after(*key*, *t_after=0*) → list
Returns buffer after t_after (unix timestamp als number). :param key: Property name like 'kelvin' or 'sensorunit' :param t_after: timstamp after which buffered values will be returned

Returns: buffered values as 2D matrix.

property internal_setpoint: float
Gets target temperature curve.

property internal_status_name: str
Gets string representation of internal status name.

property is_active: bool
Gets indicator whether controller is active.

property is_blocked: bool
Gets indicator whether device is blocked.

property is_blocked_by: str
Gets name(s) of blocking device(s) as string.

property kelvin: float
Gets measured temperature of temperature control thermometer in K.

property name: str
Gets device name.

property operation_mode: str
cadr, adr, tandem (this will be obsolete soon).

Type
Gets and sets string describing operation mode

property progress: float
Gets progress of sequence.

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

property ramp: float
Gets and sets setpoint ramp.

property ramping: bool
Gets indicator whether controller is ramping.

recharge()
Start adr recharging.

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property setpoint: float
Gets and sets setpoint.

property stable: bool
Gets indicator whether controller is stable.

start(*target: (<class 'float'>, <class 'float'>) = None*, *setpoint: float = None*, *ramp: float = None*)
Starts control sequence.

Parameters

- **target** – legacy notation for (setpoint, ramp)
- **setpoint** – Final target temperature
- **ramp** – Measurement ramp speed

start_adr(*setpoint: float*, *ramp: float*, *adr_mode: int | None = None*, *operation_mode: str | None = None*, *auto_regenerate: bool = False*, *pre_regenerate: bool = False*)
Starts adr temperature control.

Parameters

- **setpoint** – setpoint temperature to control [K]
- **ramp** – temperature ramp to control [K/min]
- **adr_mode** – ADR depth of stage to control - default is None for normal operation.
- **operation_mode** – ‘adr’ for one shot cooling and ‘cadr’ for continuous cooling.
- **auto_regenerate** – automatically regenerates in one shot mode if adr charge is empty
- **pre_regenerate** – regenerates before starting temperature control

start_cadr(*setpoint: float*, *ramp: float*, *pre_regenerate: bool = False*)

Start countinuous adr operation.

Parameters

- **setpoint** – setpoint temperature to control [K]
- **ramp** – temperature ramp to control [K/min]
- **pre_regenerate** – regenerates before starting temperature control (set to False by default)

Returns:

start_serial_adr(*setpoint: float*, *ramp: float*)

Start a serial one-shot cooldown for optimized hold-time. This will recharge the adr magnets.

Parameters

- **setpoint** – setpoint temperature to control [K]
- **ramp** – temperature ramp to control [K/min]

Returns:

start_single_adr(*setpoint: float*, *ramp: float*)

Starts adr control only using the unit connected to the samplestge for seamless control.

Parameters

- **setpoint** – setpoint temperature to control [K]
- **ramp** – temperature ramp to control [K/min]

Returns:

property state: str

Gets string representation of the controlling state-machine's current state.

property status: (<class 'int'>, <class 'str'>)

Tuple representing the device status as int and a message.

property step: str

Gets string representation of control sequence step.

stop()

Stops execution of current control sequence.

property successful_termination: bool

Gets indicator whether last controller process terminated successfully.

time() → float

Returns current system time as unix timestamp.

Returns: unix time

wait_done()

Blocks until the controller is not active anymore.

Returns:

wait_stable()

Blocks until the controller is stable. If the controller is inactive, an exception will be raised.

Returns:

3.5 Heater Control

device name: sample_heater

The heater object enables the use of a resistive sample heater. It is capable of controlling temperatures and temperature ramps starting at pre-cooling bath temperature (~4 K) up to room temperature (~300 K).

class controller_interfaces.**HeaterControl**(*device*, *host*, *port=1006*, *max_retry=10*)

Bases: TemperatureSetpointControl

Temperature control object for a resistive heater.

Use client device like illustrated below:

```
# initialize heater control client object with device object identifier and␣
↪kiutra base IP
sample_heater = HeaterControl('sample_heater', '192.168.XX.XX')
# read control object status
code, message = sample_heater.status
# reset adr control object
sample_heater.reset()
# start resistive temperature control
sample_heater.start((100., 0.25)) # --> setpoint = 100 K, ramp = 0.25 K/min)
```

property api: dict

API documentation computed at runtime based on the device implementation.

property blocking_devices: list

Gets list of device names blocking the use of the controller.

call(*key*, **args*, ***kwargs*)

Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property check_idle_state: bool

Gets indicator whether sequence is in idle state and ready to use.

property condition: str

Gets string representation of current controller condition.

property current: float

Returns measured output current.

property detailed_progress: float

Gets detailed progress of controller sequence.

property display_progress: float
Gets display progress of sequence in %.

get_buffer_after(*key*, *t_after=0*) → list
Returns buffer after t_after (unix timestamp als number). :param key: Property name like ‘kelvin’ or ‘sensorunit’ :param t_after: timstamp after which buffered values will be returned

Returns: buffered values as 2D matrix.

property input: str
Returns identifier of selected input loop

property internal_setpoint: float
Gets target temperature curve.

property internal_status_name: str
Gets string representation of internal status name.

property is_active: bool
Gets indicator whether controller is active.

property is_blocked: bool
Gets indicator whether device is blocked.

property is_blocked_by: str
Gets name(s) of blocking device(s) as string.

property kelvin: float
Gets measured temperature of temperature control thermometer in K.

property name: str
Gets device name.

property output_current: float
Returns measured output current.

property pid: (<class 'float'>, <class 'float'>, <class 'float'>)
Returns current PID parameters.

property pid_table: list
Gets specified pid table.

Sets new pid table. Format: [[T_lb, P, I, D],...]

property power: float
Returns estimated output power in W.

property progress: float
Gets progress of sequence.

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

property ramp: float
Gets and sets setpoint ramp.

property ramp_table: bool
Gets and sets indicator whether to use pid table.

property ramping: bool
Gets indicator whether controller is ramping.

property range: str
Returns active heater range.

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property setpoint: float
Gets and sets setpoint.

property stable: bool
Gets indicator whether controller is stable.

start(*target: (<class 'float'>, <class 'float'>) = None*, *setpoint: float = None*, *ramp: float = None*)
Starts control sequence.

Parameters

- **target** – legacy notation for (setpoint, ramp)
- **setpoint** – Final target temperature
- **ramp** – Measurement ramp speed

property state: str
Gets string representation of the controlling state-machine's current state.

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

property step: str
Gets string representation of control sequence step.

stop()
Stops execution of current control sequence.

property successful_termination: bool
Gets indicator whether last controller process terminated successfully.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

wait_done()
Blocks until the controller is not active anymore.

Returns:

wait_stable()
Blocks until the controller is stable. If the controller is inactive, an exception will be raised.

Returns:

3.6 Magnet Control

device name: sample_magnet

The sample magnet control object is a dedicated object to control (optional) sample magnets. It controls and monitors the sample magnets field and field ramps.

class controller_interfaces.**MagnetControl**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: SetpointControl

Magnet control object for a super conducting sample magnet.

Use client device like illustrated below:

```
# initialize magnet control client object with device object identifier and␣
↪kiutra base IP
sample_magnet = MagnetControl('sample_magnet', '192.168.XX.XX')
# read control object status
code, message = sample_magnet.status
# reset magnet control object
sample_magnet.reset()
# start magnet control
sample_magnet.start((3., 0.25)) # --> setpoint = 3 T, ramp = 0.25 T/min
```

property api: dict
API documentation computed at runtime based on the device implementation.

property blocking_devices: list
Gets list of device names blocking the use of the controller.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property check_idle_state: bool
Gets indicator whether sequence is in idle state and ready to use.

property condition: str
Gets string representation of current controller condition.

property current: float
Reads power supply current.

property current_buffer: list
Reads the magnet's buffered current.

property current_median: float
Reads the magnet's current median in "transientwindow".

property current_rate: float
Reads the magnet's estimated linear current rate over "transientwindow".

property current_setpoint
Gets current setpoint in A.

property current_stabilitywindow: float
Reads length of current stabilitywindow in sec.

property current_stablebuffer: list
Reads the magnet's buffered current in "stabilitywindow.

property current_timestamp: float
Reads tuple of magnet's current and time in sec. (current, timestamp).

property current_transientbuffer: list
Reads the magnet's buffered current in "transientwindow.

property current_transientwindow: float
Reads length of current transientwindow in sec.

property detailed_progress: float
Gets detailed progress of controller sequence.

property display_progress: float
Gets display progress of sequence in %.

property field
Gets field in T.

property field_rate
Gets field rate setpoint in T/min.

property field_setpoint
Gets field setpoint in T.

get_buffer_after(*key*, *t_after=0*) → list
Returns buffer after t_after (unix timestamp als number). :param key: Property name like 'kelvin' or 'sensorunit' :param t_after: timstamp after which buffered values will be returned

Returns: buffered values as 2D matrix.

property internal_setpoint: float
Gets target temperature curve.

property internal_status_name: str
Gets string representation of internal status name.

property is_active: bool
Gets indicator whether controller is active.

property is_blocked: bool
Gets indicator whether device is blocked.

property is_blocked_by: str
Gets name(s) of blocking device(s) as string.

property name: str
Gets device name.

off()
Switch magnet power supply(s) off

on()
Switch magnet power supply(s) on

property power
Gets output power in W.

property progress: float
Gets progress of sequence.

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

property ramp: float
Gets and sets setpoint ramp.

property ramping: bool
Gets indicator whether controller is ramping.

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property setpoint: float
Gets and sets setpoint.

property stable: bool
Gets indicator whether controller is stable.

start(*target: (<class 'float'>, <class 'float'>) = None*, *setpoint: float = None*, *ramp: float = None*)
Starts control sequence.

Parameters

- **target** – legacy notation for (setpoint, ramp)
- **setpoint** – Final target temperature

- **ramp** – Measurement ramp speed

property state: str
Gets string representation of the controlling state-machine's current state.

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

property step: str
Gets string representation of control sequence step.

stop()
Stops execution of current control sequence.

property successful_termination: bool
Gets indicator whether last controller process terminated successfully.

property tesla: float
Reads power supply tesla.

property tesla_buffer: list
Reads the magnet's buffered tesla.

property tesla_median: float
Reads the magnet's tesla median in "transientwindow".

property tesla_rate: float
Reads the magnet's estimated linear tesla rate over "transientwindow".

property tesla_stabilitywindow: float
Reads length of tesla stabilitywindow in sec.

property tesla_stablebuffer: list
Reads the magnet's buffered tesla in "stabilitywindow.

property tesla_timestamp: float
Reads tuple of magnet's tesla and time in sec. (tesla, timestamp).

property tesla_transientbuffer: list
Reads the magnet's buffered tesla in "transientwindow.

property tesla_transientwindow: float
Reads length of tesla transientwindow in sec.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

property voltage
Gets voltage current in V.

wait_done()
Blocks until the controller is not active anymore.

Returns:

wait_stable()

Blocks until the controller is stable. If the controller is inactive, an exception will be raised.

Returns:

3.7 Utility Control

device: utility_control

The utility control object offers several utility functions to carry out basic operations securely. For instance, opening critical valves without checking the system status might damage the cryostat. The utility control checks these metrics before executing a critical operation.

class controller_interfaces.**Control**(*device*, *host*, *port=1006*, *max_retry=10*)

Bases: Device

property api: dict

API documentation computed at runtime based on the device implementation.

property blocking_devices: list

Gets list of device names blocking the use of the controller.

call(*key*, **args*, ***kwargs*)

Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property check_idle_state: bool

Gets indicator whether sequence is in idle state and ready to use.

property detailed_progress: float

Gets detailed progress of controller sequence.

property display_progress: float

Gets display progress of sequence in %.

property internal_status_name: str

Gets string representation of internal status name.

property is_active: bool

Gets indicator whether controller is active.

property is_blocked: bool

Gets indicator whether device is blocked.

property is_blocked_by: str

Gets name(s) of blocking device(s) as string.

property name: str

Gets device name.

property progress: float
Gets progress of sequence.

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

start(*target: str = 'default'*)
Starts control sequence.

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

property step: str
Gets string representation of control sequence step.

stop()
Stops execution of current control sequence.

property successful_termination: bool
Gets indicator whether last controller process terminated successfully.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

CHAPTER 4

Devices

Low-Level Devices

Attention

Admin-level access is required to change low-level device parameter or execute functions.

Note

You can read device metrics without admin-level access.

Low-level devices represent crucial components of the cryostat. To operate the cryostat no interaction with these devices is required. For monitoring, however, device-level metrics can be read by any user.

4.1 Heatswitch

device name: hs1, hs2, .., hs_sample

class device_interfaces.**Heatswitch**(*device*, *host*, *port=1006*, *max_retry=10*)

Bases: Device

property api: dict

API documentation computed at runtime based on the device implementation.

call(*key*, **args*, ***kwargs*)

Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

closed()
Close heat switch.

property is_closed: bool
Indicates whether the heat switch is closed.

property is_open: bool
Indicates whether the heat switch is open.

open()
Open heat switch.

property position: float
Reads the heat switch position (0-100%).

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property setpoint: str
Reads and sets the heat switch's target setpoint.

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

4.2 Pressure Regulator

device names: pressure_regulator

class device_interfaces.**PressureRegulator**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: Device

property api: dict
API documentation computed at runtime based on the device implementation.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property pressure
Reads current pressure in mbar/bar.

property pressure_setpoint
Reads current pressure setpoint in mbar/bar.

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

property voltage
Reads raw sensor voltage.

4.3 Thermometer

device name: T_system, T_sample, T_ccr1, T_ccr2, T_adr1, T_adr2, .., T_adrn, T_lazy

class device_interfaces.**Thermometer**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: Device

property api: dict
API documentation computed at runtime based on the device implementation.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property excitation: float

Reads the thermometer's excitation in V.

get_buffer_after(*key*, *t_after=0*) → list

Returns buffer after t_after (unix timestamp als number). :param key: Property name like 'kelvin' or 'sensorunit' :param t_after: timstamp after which buffered values will be returned

Returns: buffered values as 2D matrix.

get_settled(*max_error=None*, *setpoint=None*) → bool

Returns whether temperature is settled with respect to setpoint. Settled requires all relative errors delta_T/T in "stabilitywindow" to be within an epsilon of +-max_error.

Parameters

- **max_error** – Maximum relative temperature error.
- **setpoint** – Setpoint value to compare to.

Returns

Indicator whether temperature is settled.

get_settling(*max_error=None*, *setpoint=None*) → bool

Returns whether temperature is settling with respect to setpoint. Settling requires at least one relative error delta_T/T in "stabilitywindow" to be within an epsilon of +-max_error.

Parameters

- **max_error** – Maximum relative temperature error.
- **setpoint** – Setpoint value to compare to.

Returns

Indicator whether temperature is settling.

get_stable(*max_error=None*, *setpoint=None*) → list

Returns list of indicators for alle values within "stabilitywindow". If a value meets the stability criterion, the respective entry is True, else False. A value becomes True if delta_T/T < max_error where delta_T = abs(T-setpoint)

Parameters

- **max_error** – Maximum relative temperature error.
- **setpoint** – Setpoint value to compare to.

Returns

List of boolean indicators whether an entry reaches the stability criterion.

property kelvin: float

Reads the thermometer's temperature in K.

property kelvin_buffer: list

Reads the thermometer's buffered temperatures.

property kelvin_median: float

Reads the thermometer's temperature median in "transientwindow".

property kelvin_rate: float

Reads the thermometer's estimated linear temperature rate in K/s over "transientwindow".

property kelvin_stabilitywindow: float

Reads length of temperature stabilitywindow in sec.

property kelvin_stablebuffer: list

Reads the thermometer's buffered temperatures in "stabilitywindow.

property kelvin_timestamp: float

Reads tuple of thermometer's temperature in K and time in sec. (temperature, timestamp).

property kelvin_transientbuffer: list

Reads the thermometer's buffered temperatures in "transientwindow.

property kelvin_transientwindow: float

Reads length of temperature transientwindow in sec.

property max_error: float

abs(setpoint-temperature)/temperature < max_error.

Type

Reads allowed maximum relative error of thermometer

query(*key*, **args*)

Queries a parameter based on key.

Parameters

key – parameter handle like device.parameter

reset()

Resets device.

property sensorunits: float

Reads the thermometer's temperature in sensor units (normally Ohm).

property sensorunits_buffer: list

Reads the thermometer's buffered sensorunits.

property sensorunits_median: float

Reads the thermometer's sensorunits median in "transientwindow".

property sensorunits_rate: float

Reads the thermometer's estimated linear sensorunits rate over "transientwindow".

property sensorunits_stabilitywindow: float

Reads length of sensorunits stabilitywindow in sec.

property sensorunits_stablebuffer: list

Reads the thermometer's buffered sensorunits in "stabilitywindow.

property sensorunits_timestamp: float

Reads tuple of thermometer's sensorunits and time in sec. (sensorunits, timestamp).

property sensorunits_transientbuffer: list

Reads the thermometer's buffered sensorunits in "transientwindow.

property sensorunits_transientwindow: float
Reads length of sensorunits transientwindow in sec.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

4.4 Magnet Power Supply

device name: mps_1, mps_2, ..., mps_n, mag_sample

class device_interfaces.**Magnet**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: Device

property api: dict
API documentation computed at runtime based on the device implementation.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property current: float
Reads power supply current.

property current_buffer: list
Reads the magnet's buffered current.

property current_median: float
Reads the magnet's current median in "transientwindow".

property current_rate: float
Reads the magnet's estimated linear current rate over "transientwindow".

property current_stabilitywindow: float
Reads length of current stabilitywindow in sec.

property current_stablebuffer: list
Reads the magnet's buffered current in "stabilitywindow.

property current_timestamp: float

Reads tuple of magnet's current and time in sec. (current, timestamp).

property current_transientbuffer: list

Reads the magnet's buffered current in "transientwindow.

property current_transientwindow: float

Reads length of current transientwindow in sec.

property field: float

Reads power supply field.

property field_rate: float

Reads power supply field.

get_buffer_after(*key*, *t_after=0*) → list

Returns buffer after t_after (unix timestamp als number). :param key: Property name like 'kelvin' or 'sensorunit' :param t_after: timstamp after which buffered values will be returned

Returns: buffered values as 2D matrix.

get_settled(*max_error=None*, *setpoint=None*) → bool

Returns whether field is settled with respect to setpoint. Settled requires all relative errors delta_B/B in "stabilitywindow" to be within an epsilon of +-max_error.

Parameters

- **max_error** – Maximum relative field error.
- **setpoint** – Setpoint value to compare to.

Returns

Indicator whether field is settled.

get_settling(*max_error=None*, *setpoint=None*) → bool

Returns whether field is settling with respect to setpoint. Settling requires at least one relative error delta_B/B in "stabilitywindow" to be within an epsilon of +-max_error.

Parameters

- **max_error** – Maximum relative field error.
- **setpoint** – Setpoint value to compare to.

Returns

Indicator whether field is settling.

get_stable(*max_error=None*, *setpoint=None*) → list

Returns list of indicators for alle values within "stabilitywindow". If a value meets the stability criterion, the respective entry is True, else False. A value becomes True if delta_B/B < max_error where delta_B = abs(B-setpoint)

Parameters

- **max_error** – Maximum relative temperature error.
- **setpoint** – Setpoint value to compare to.

Returns

List of boolean indicators whether an entry reaches the stability criterion.

property max_error: float
abs(setpoint-field)/field < max_error.

Type
Reads allowed maximum relative error of magnet

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

property tesla: float
Reads power supply tesla.

property tesla_buffer: list
Reads the magnet's buffered tesla.

property tesla_median: float
Reads the magnet's tesla median in "transientwindow".

property tesla_rate: float
Reads the magnet's estimated linear tesla rate over "transientwindow".

property tesla_stabilitywindow: float
Reads length of tesla stabilitywindow in sec.

property tesla_stablebuffer: list
Reads the magnet's buffered tesla in "stabilitywindow.

property tesla_timestamp: float
Reads tuple of magnet's tesla and time in sec. (tesla, timestamp).

property tesla_transientbuffer: list
Reads the magnet's buffered tesla in "transientwindow.

property tesla_transientwindow: float
Reads length of tesla transientwindow in sec.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

property voltage: float
Reads power supply voltage.

property voltage_rate: float
Reads power supply voltage rate.

4.5 Baffles

device name: baffles

class device_interfaces.**Baffles**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: Device

property api: dict
API documentation computed at runtime based on the device implementation.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property is_closed: bool
Indicates whether the baffles are closed.

property is_open: bool
Indicates whether the baffles are open.

property position: float
Reads the heat baffles position (0-100%).

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property setpoint: str
Reads and sets the baffles' target setpoint.

> **Note**
>
> Admin-level access required.

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

stop()
Stops baffles.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

4.6 Valve

device names: purge, utility, main

class device_interfaces.**Valve**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: Device

property api: dict
API documentation computed at runtime based on the device implementation.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

closed()
Close valve.

> **Note**
>
> Admin-level access required.

property is_closed: bool
Indicates whether the valve is closed.

property is_open: bool
Indicates whether the valve is open.

property position: str
Returns a string representation of the valve's position.

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

4.7 Winch

device name: winch

class device_interfaces.**Winch**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: Device

property api: dict
API documentation computed at runtime based on the device implementation.

property bellows_is_down: bool
Indicates whether the the bellows is down.

property bellows_is_up: bool
Indicates whether the the bellows is up.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property door_is_closed: bool
Indicates whether the airlock door is closed.

property door_is_open: bool
Indicates whether the airlock door is open.

home()
Homes the motor position.

> **Note**
> Admin-level access required.

property position: float
Reads the motor position (0-100%)

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

property raw_position: float
Reads the motor's raw position.

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property setpoint: str
Reads the motor's setpoint (up/down).

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

stop()
Stops the motor immediately.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

4.8 Warmup Heater

device name: warmup_heter

class device_interfaces.**WarmupHeater**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: Device

property api: dict
API documentation computed at runtime based on the device implementation.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property current: float
Reads the heater's output current in A.

property input: float
Reads the selected input channel.

property kelvin: float
Reads the control thermometer's temperature in K.

off()
Switches heating loop to mode off.

property power: float
Reads the heater's power consumption in W.

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

property range: float
Reads the selected heater range.

property rate: float
Reads the heater's temperature ramp rate in K/min.

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property setpoint: float
Reads the heater's temperature setpoint in K.

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

stop()
Stops current heating process.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

property voltage: float
Reads the heater's output voltage in V.

4.9 Gauge

device names: isovac, inlet, compressed_air, purge_gas

class device_interfaces.**Gauge**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: Device

property api: dict
API documentation computed at runtime based on the device implementation.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property pressure
Reads current pressure in mbar/bar.

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

property voltage
Reads raw sensor voltage.

4.10 Compressor

device name: compressor

class device_interfaces.**Compressor**(*device*, *host*, *port=1006*, *max_retry=10*)

Bases: Device

property api: dict

API documentation computed at runtime based on the device implementation.

call(*key*, **args*, ***kwargs*)

Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property gasflow_error: bool

Indicates a motor gas flow error.

property gastemp_error: bool

Indicates a motor gas temperature error.

property motortemp_error: bool

Indicates a motor temperature error.

off()

Switches the compressor off.

on()

Switches the compressor on.

property operating_hours: float

Reads and sets operating hours as stored on the compressor control unit (CCU).

property power_error: bool

Indicates a power temperature error.

query(*key*, **args*)

Queries a parameter based on key.

Parameters

key – parameter handle like device.parameter

reset()

Resets device.

property return_pressure: float

Reads the compressor return pressure in bar.

property runstatus

Indicates whether the compressor is running.

set(*key*, *value*)

Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

property voltage: float
Reads the raw compressor return pressure voltage.

property waterflow_error: bool
Indicates a motor water flow error.

property watertemp_error: bool
Indicates a motor water temperature error.

4.11 Pumps

device name: turbopump

class device_interfaces.**TurboPump**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: Device

property api: dict
API documentation computed at runtime based on the device implementation.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function
- **args** – arguments

property current: float
Pump's motor current draw in A.

property frequency: float
Pump's current rotor frequency in Hz.

property is_off: bool
Indicates whether the pump is switched off.

property is_on: bool
Indicates whether the pump is drawing power.

property is_rotating: bool
Indicates whether the pump's rotor is spinning.

property operation_counter: int
Number of starts.

property operation_hours: float
Total operation hours.

property power: float
Pump's motor power consumption in W.

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

stop()
Stop turbo pump.

Note
Admin-level access required.

property temperature: float
Pump's motor temperature in °C.

time() → float
Returns current system time as unix timestamp.

Returns: unix time

property voltage: float
Pump's motor voltage draw in V.

device name: forepump

class device_interfaces.**ForePump**(*device*, *host*, *port=1006*, *max_retry=10*)
Bases: Device

property api: dict
API documentation computed at runtime based on the device implementation.

call(*key*, **args*, ***kwargs*)
Calls a remote procedure function based on a key and passes arguments.

Parameters

- **key** – function handle like device.function

- **args** – arguments

property is_off: bool
Indicates whether the pump is switched off.

property is_on: bool
Indicates whether the pump is drawing power.

off()
Switches the pump to off.

> **Note**
> Admin-level access required.

on()
Switches the pump to on.

> **Note**
> Admin-level access required.

query(*key*, **args*)
Queries a parameter based on key.

Parameters
key – parameter handle like device.parameter

reset()
Resets device.

set(*key*, *value*)
Sets a parameter based on key to value.

Parameters

- **key** – parameter handle like device.parameter
- **value** – target value

start()
Start Fore pump.

> **Note**
> Admin-level access required.

property status: (<class 'int'>, <class 'str'>)
Tuple representing the device status as int and a message.

stop()
Stop fore pump.

> **Note**
>
> Admin-level access required.

time() → float

Returns current system time as unix timestamp.

Returns: unix time

Index

A

access_level (*api_client.APIManager property*), 7
adr_mode (*controller_interfaces.ADRControl property*), 21
adr_pid (*controller_interfaces.TemperatureControl property*), 16
adr_pid_table (*controller_interfaces.TemperatureControl property*), 16
ADRControl (*class in controller_interfaces*), 21
api (*api_client.APIManager property*), 7
api (*controller_interfaces.ADRControl property*), 21
api (*controller_interfaces.Control property*), 32
api (*controller_interfaces.CryostatControl property*), 10
api (*controller_interfaces.HeaterControl property*), 25
api (*controller_interfaces.MagnetControl property*), 28
api (*controller_interfaces.SampleControl property*), 13
api (*controller_interfaces.TemperatureControl property*), 16
api (*device_interfaces.Baffles property*), 43
api (*device_interfaces.Compressor property*), 49
api (*device_interfaces.ForePump property*), 51
api (*device_interfaces.Gauge property*), 48
api (*device_interfaces.Heatswitch property*), 35
api (*device_interfaces.Magnet property*), 40
api (*device_interfaces.PressureRegulator property*), 36
api (*device_interfaces.Thermometer property*), 37
api (*device_interfaces.TurboPump property*), 50
api (*device_interfaces.Valve property*), 44
api (*device_interfaces.WarmupHeater property*), 46
api (*device_interfaces.Winch property*), 45
api_info (*api_client.APIManager property*), 7
APIManager (*class in api_client*), 7

B

Baffles (*class in device_interfaces*), 43
bellows_is_down (*device_interfaces.Winch property*), 45
bellows_is_up (*device_interfaces.Winch property*), 45
blocking_devices (*controller_interfaces.ADRControl property*), 21
blocking_devices (*controller_interfaces.Control property*), 32
blocking_devices (*controller_interfaces.CryostatControl property*), 10
blocking_devices (*controller_interfaces.HeaterControl property*), 25
blocking_devices (*controller_interfaces.MagnetControl property*), 28
blocking_devices (*controller_interfaces.SampleControl property*), 14
blocking_devices (*controller_interfaces.TemperatureControl property*), 16

C

call() (*api_client.APIManager method*), 7
call() (*api_client.KiutraClient method*), 6
call() (*controller_interfaces.ADRControl method*), 21
call() (*controller_interfaces.Control method*), 32
call() (*controller_interfaces.CryostatControl method*), 10
call() (*controller_interfaces.HeaterControl method*), 25
call() (*controller_interfaces.MagnetControl method*), 28
call() (*controller_interfaces.SampleControl method*), 14
call() (*controller_interfaces.TemperatureControl method*), 16
call() (*device_interfaces.Baffles method*), 43
call() (*device_interfaces.Compressor method*), 49
call() (*device_interfaces.ForePump method*), 51
call() (*device_interfaces.Gauge method*), 48
call() (*device_interfaces.Heatswitch method*), 35
call() (*device_interfaces.Magnet method*), 40
call() (*device_interfaces.PressureRegulator method*), 36
call() (*device_interfaces.Thermometer method*), 37
call() (*device_interfaces.TurboPump method*), 50
call() (*device_interfaces.Valve method*), 44
call() (*device_interfaces.WarmupHeater method*), 46
call() (*device_interfaces.Winch method*), 45
charge (*controller_interfaces.ADRControl property*), 21
check_idle_state (*controller_interfaces.ADRControl property*), 22
check_idle_state (*controller_interfaces.Control property*), 32
check_idle_state (*controller_interfaces.CryostatControl property*), 10
check_idle_state (*controller_interfaces.HeaterControl property*), 25
check_idle_state (*controller_interfaces.MagnetControl property*), 28
check_idle_state (*controller_interfaces.SampleControl property*), 14
check_idle_state (*controller_interfaces.TemperatureControl property*), 17

check_magnets() (*controller_interfaces.CryostatControl method*), 10
clear_magnets() (*controller_interfaces.CryostatControl method*), 10
close_airlock() (*controller_interfaces.SampleControl method*), 14
close_baffles() (*controller_interfaces.CryostatControl method*), 10
close_gate() (*controller_interfaces.CryostatControl method*), 10
closed() (*device_interfaces.Heatswitch method*), 36
closed() (*device_interfaces.Valve method*), 44
Compressor (*class in device_interfaces*), 49
condition (*controller_interfaces.ADRControl property*), 22
condition (*controller_interfaces.HeaterControl property*), 25
condition (*controller_interfaces.MagnetControl property*), 28
condition (*controller_interfaces.TemperatureControl property*), 17
Control (*class in controller_interfaces*), 32
cooldown() (*controller_interfaces.CryostatControl method*), 11
cooldown() (*controller_interfaces.TemperatureControl method*), 17
cooldown_block() (*controller_interfaces.CryostatControl method*), 11
cooldown_only() (*controller_interfaces.CryostatControl method*), 11
CryostatControl (*class in controller_interfaces*), 10
current (*controller_interfaces.HeaterControl property*), 25
current (*controller_interfaces.MagnetControl property*), 28
current (*device_interfaces.Magnet property*), 40
current (*device_interfaces.TurboPump property*), 50
current (*device_interfaces.WarmupHeater property*), 47
current_buffer (*controller_interfaces.MagnetControl property*), 28
current_buffer (*device_interfaces.Magnet property*), 40
current_median (*controller_interfaces.MagnetControl property*), 29
current_median (*device_interfaces.Magnet property*), 40
current_rate (*controller_interfaces.MagnetControl property*), 29
current_rate (*device_interfaces.Magnet property*), 40
current_setpoint (*controller_interfaces.MagnetControl property*), 29
current_stabilitywindow (*controller_interfaces.MagnetControl property*), 29
current_stabilitywindow (*device_interfaces.Magnet property*), 40
current_stablebuffer (*controller_interfaces.MagnetControl property*), 29
current_stablebuffer (*device_interfaces.Magnet property*), 40
current_timestamp (*controller_interfaces.MagnetControl property*), 29
current_timestamp (*device_interfaces.Magnet property*), 40
current_transientbuffer (*controller_interfaces.MagnetControl property*), 29
current_transientbuffer (*device_interfaces.Magnet property*), 41
current_transientwindow (*controller_interfaces.MagnetControl property*), 29
current_transientwindow (*device_interfaces.Magnet property*), 41

D

detailed_progress (*controller_interfaces.ADRControl property*), 22
detailed_progress (*controller_interfaces.Control property*), 32
detailed_progress (*controller_interfaces.CryostatControl property*), 11
detailed_progress (*controller_interfaces.HeaterControl property*), 25
detailed_progress (*controller_interfaces.MagnetControl property*), 29
detailed_progress (*controller_interfaces.SampleControl property*), 14
detailed_progress (*controller_interfaces.TemperatureControl property*), 17
device_check() (*controller_interfaces.CryostatControl method*), 11
display_progress (*controller_interfaces.ADRControl property*), 22
display_progress (*controller_interfaces.Control property*), 32
display_progress (*controller_interfaces.CryostatControl property*), 11
display_progress (*controller_interfaces.HeaterControl property*), 25
display_progress (*controller_interfaces.MagnetControl property*), 29
display_progress (*controller_interfaces.SampleControl property*), 14
display_progress (*controller_interfaces.TemperatureControl property*), 17
door_is_closed (*device_interfaces.Winch property*), 45
door_is_open (*device_interfaces.Winch property*), 45
door_open (*controller_interfaces.SampleControl property*), 14

E

evacuate() (*controller_interfaces.CryostatControl method*), 11
excitation (*device_interfaces.Thermometer property*), 38

F

field (*controller_interfaces.MagnetControl property*), 29
field (*device_interfaces.Magnet property*), 41
field_rate (*controller_interfaces.MagnetControl property*), 29
field_rate (*device_interfaces.Magnet property*), 41
field_setpoint (*controller_interfaces.MagnetControl property*), 29

ForePump (*class in device_interfaces*), 51
frequency (*device_interfaces.TurboPump property*), 50

G

gasflow_error (*device_interfaces.Compressor property*), 49
gastemp_error (*device_interfaces.Compressor property*), 49
Gauge (*class in device_interfaces*), 48
get_buffer_after() (*controller_interfaces.ADRControl method*), 22
get_buffer_after() (*controller_interfaces.HeaterControl method*), 26
get_buffer_after() (*controller_interfaces.MagnetControl method*), 29
get_buffer_after() (*controller_interfaces.TemperatureControl method*), 17
get_buffer_after() (*device_interfaces.Magnet method*), 41
get_buffer_after() (*device_interfaces.Thermometer method*), 38
get_ramping_info() (*controller_interfaces.TemperatureControl method*), 17
get_settled() (*device_interfaces.Magnet method*), 41
get_settled() (*device_interfaces.Thermometer method*), 38
get_settling() (*device_interfaces.Magnet method*), 41
get_settling() (*device_interfaces.Thermometer method*), 38
get_stable() (*device_interfaces.Magnet method*), 41
get_stable() (*device_interfaces.Thermometer method*), 38

H

heater_pid (*controller_interfaces.TemperatureControl property*), 17
heater_pid_table (*controller_interfaces.TemperatureControl property*), 17
HeaterControl (*class in controller_interfaces*), 25
Heatswitch (*class in device_interfaces*), 35
home() (*device_interfaces.Winch method*), 45

I

initialize() (*controller_interfaces.CryostatControl method*), 11
input (*controller_interfaces.HeaterControl property*), 26
input (*device_interfaces.WarmupHeater property*), 47
internal_setpoint (*controller_interfaces.ADRControl property*), 22
internal_setpoint (*controller_interfaces.HeaterControl property*), 26
internal_setpoint (*controller_interfaces.MagnetControl property*), 29
internal_setpoint (*controller_interfaces.TemperatureControl property*), 17
internal_status_name (*controller_interfaces.ADRControl property*), 22
internal_status_name (*controller_interfaces.Control property*), 32
internal_status_name (*controller_interfaces.CryostatControl property*), 11
internal_status_name (*controller_interfaces.HeaterControl property*), 26
internal_status_name (*controller_interfaces.MagnetControl property*), 29
internal_status_name (*controller_interfaces.SampleControl property*), 14
internal_status_name (*controller_interfaces.TemperatureControl property*), 17
is_active (*controller_interfaces.ADRControl property*), 22
is_active (*controller_interfaces.Control property*), 32
is_active (*controller_interfaces.CryostatControl property*), 11
is_active (*controller_interfaces.HeaterControl property*), 26
is_active (*controller_interfaces.MagnetControl property*), 29
is_active (*controller_interfaces.SampleControl property*), 14
is_active (*controller_interfaces.TemperatureControl property*), 17
is_blocked (*controller_interfaces.ADRControl property*), 22
is_blocked (*controller_interfaces.Control property*), 32
is_blocked (*controller_interfaces.CryostatControl property*), 11
is_blocked (*controller_interfaces.HeaterControl property*), 26
is_blocked (*controller_interfaces.MagnetControl property*), 29
is_blocked (*controller_interfaces.SampleControl property*), 14
is_blocked (*controller_interfaces.TemperatureControl property*), 17
is_blocked_by (*controller_interfaces.ADRControl property*), 22
is_blocked_by (*controller_interfaces.Control property*), 32
is_blocked_by (*controller_interfaces.CryostatControl property*), 11
is_blocked_by (*controller_interfaces.HeaterControl property*), 26
is_blocked_by (*controller_interfaces.MagnetControl property*), 30
is_blocked_by (*controller_interfaces.SampleControl property*), 14
is_blocked_by (*controller_interfaces.TemperatureControl property*), 17
is_closed (*device_interfaces.Baffles property*), 43
is_closed (*device_interfaces.Heatswitch property*), 36
is_closed (*device_interfaces.Valve property*), 44
is_off (*device_interfaces.ForePump property*), 52
is_off (*device_interfaces.TurboPump property*), 50
is_on (*device_interfaces.ForePump property*), 52
is_on (*device_interfaces.TurboPump property*), 50
is_open (*device_interfaces.Baffles property*), 43
is_open (*device_interfaces.Heatswitch property*), 36
is_open (*device_interfaces.Valve property*), 44
is_rotating (*device_interfaces.TurboPump property*), 50

K

kelvin (*controller_interfaces.ADRControl property*), 22
kelvin (*controller_interfaces.CryostatControl property*), 11
kelvin (*controller_interfaces.HeaterControl property*), 26
kelvin (*controller_interfaces.TemperatureControl property*), 17
kelvin (*device_interfaces.Thermometer property*), 38
kelvin (*device_interfaces.WarmupHeater property*), 47
kelvin_buffer (*device_interfaces.Thermometer property*), 38
kelvin_median (*device_interfaces.Thermometer property*), 38
kelvin_rate (*device_interfaces.Thermometer property*), 38
kelvin_stabilitywindow (*device_interfaces.Thermometer property*), 39
kelvin_stablebuffer (*device_interfaces.Thermometer property*), 39
kelvin_timestamp (*device_interfaces.Thermometer property*), 39
kelvin_transientbuffer (*device_interfaces.Thermometer property*), 39
kelvin_transientwindow (*device_interfaces.Thermometer property*), 39
KiutraClient (*class in api_client*), 5

L

load_sample() (*controller_interfaces.SampleControl method*), 14
lock() (*api_client.APIManager method*), 7

M

Magnet (*class in device_interfaces*), 40
MagnetControl (*class in controller_interfaces*), 28
max_error (*device_interfaces.Magnet property*), 41
max_error (*device_interfaces.Thermometer property*), 39
motortemp_error (*device_interfaces.Compressor property*), 49

N

name (*controller_interfaces.ADRControl property*), 22
name (*controller_interfaces.Control property*), 32
name (*controller_interfaces.CryostatControl property*), 11
name (*controller_interfaces.HeaterControl property*), 26
name (*controller_interfaces.MagnetControl property*), 30
name (*controller_interfaces.SampleControl property*), 14
name (*controller_interfaces.TemperatureControl property*), 17

O

off() (*controller_interfaces.MagnetControl method*), 30
off() (*device_interfaces.Compressor method*), 49
off() (*device_interfaces.ForePump method*), 52
off() (*device_interfaces.WarmupHeater method*), 47
on() (*controller_interfaces.MagnetControl method*), 30
on() (*device_interfaces.Compressor method*), 49
on() (*device_interfaces.ForePump method*), 52
open() (*device_interfaces.Heatswitch method*), 36
open_airlock() (*controller_interfaces.SampleControl method*), 14
open_baffles() (*controller_interfaces.CryostatControl method*), 11
open_gate() (*controller_interfaces.CryostatControl method*), 11
operating_hours (*device_interfaces.Compressor property*), 49
operation_counter (*device_interfaces.TurboPump property*), 50
operation_hours (*device_interfaces.TurboPump property*), 50
operation_mode (*controller_interfaces.ADRControl property*), 22
output_current (*controller_interfaces.HeaterControl property*), 26

P

pid (*controller_interfaces.HeaterControl property*), 26
pid_table (*controller_interfaces.HeaterControl property*), 26
position (*device_interfaces.Baffles property*), 43
position (*device_interfaces.Heatswitch property*), 36
position (*device_interfaces.Valve property*), 44
position (*device_interfaces.Winch property*), 46
power (*controller_interfaces.HeaterControl property*), 26
power (*controller_interfaces.MagnetControl property*), 30
power (*device_interfaces.TurboPump property*), 51
power (*device_interfaces.WarmupHeater property*), 47
power_error (*device_interfaces.Compressor property*), 49
pressure (*controller_interfaces.CryostatControl property*), 11
pressure (*device_interfaces.Gauge property*), 48
pressure (*device_interfaces.PressureRegulator property*), 37
pressure_setpoint (*device_interfaces.PressureRegulator property*), 37
PressureRegulator (*class in device_interfaces*), 36
progress (*controller_interfaces.ADRControl property*), 22
progress (*controller_interfaces.Control property*), 32
progress (*controller_interfaces.CryostatControl property*), 11
progress (*controller_interfaces.HeaterControl property*), 26
progress (*controller_interfaces.MagnetControl property*), 30
progress (*controller_interfaces.SampleControl property*), 14
progress (*controller_interfaces.TemperatureControl property*), 18
propose_control_mode() (*controller_interfaces.TemperatureControl method*), 18
pump_airlock() (*controller_interfaces.CryostatControl method*), 11
pump_airlock() (*controller_interfaces.SampleControl method*), 14
pump_suspend() (*controller_interfaces.CryostatControl method*), 12
purge() (*controller_interfaces.CryostatControl method*), 12
purge_airlock() (*controller_interfaces.SampleControl method*), 14

Q

query() (*api_client.APIManager method*), 7
query() (*api_client.KiutraClient method*), 6
query() (*controller_interfaces.ADRControl method*), 22
query() (*controller_interfaces.Control method*), 33

query() (*controller_interfaces.CryostatControl method*), 12
query() (*controller_interfaces.HeaterControl method*), 26
query() (*controller_interfaces.MagnetControl method*), 30
query() (*controller_interfaces.SampleControl method*), 15
query() (*controller_interfaces.TemperatureControl method*), 18
query() (*device_interfaces.Baffles method*), 43
query() (*device_interfaces.Compressor method*), 49
query() (*device_interfaces.ForePump method*), 52
query() (*device_interfaces.Gauge method*), 48
query() (*device_interfaces.Heatswitch method*), 36
query() (*device_interfaces.Magnet method*), 42
query() (*device_interfaces.PressureRegulator method*), 37
query() (*device_interfaces.Thermometer method*), 39
query() (*device_interfaces.TurboPump method*), 51
query() (*device_interfaces.Valve method*), 44
query() (*device_interfaces.WarmupHeater method*), 47
query() (*device_interfaces.Winch method*), 46

R

ramp (*controller_interfaces.ADRControl property*), 22
ramp (*controller_interfaces.HeaterControl property*), 26
ramp (*controller_interfaces.MagnetControl property*), 30
ramp (*controller_interfaces.TemperatureControl property*), 18
ramp_table (*controller_interfaces.HeaterControl property*), 26
ramping (*controller_interfaces.ADRControl property*), 23
ramping (*controller_interfaces.HeaterControl property*), 27
ramping (*controller_interfaces.MagnetControl property*), 30
ramping (*controller_interfaces.TemperatureControl property*), 18
range (*controller_interfaces.HeaterControl property*), 27
range (*device_interfaces.WarmupHeater property*), 47
rate (*device_interfaces.WarmupHeater property*), 47
raw_position (*device_interfaces.Winch property*), 46
recharge() (*controller_interfaces.ADRControl method*), 23
recharge() (*controller_interfaces.TemperatureControl method*), 18
reset() (*api_client.APIManager method*), 8
reset() (*controller_interfaces.ADRControl method*), 23
reset() (*controller_interfaces.Control method*), 33
reset() (*controller_interfaces.CryostatControl method*), 12
reset() (*controller_interfaces.HeaterControl method*), 27
reset() (*controller_interfaces.MagnetControl method*), 30
reset() (*controller_interfaces.SampleControl method*), 15
reset() (*controller_interfaces.TemperatureControl method*), 18
reset() (*device_interfaces.Baffles method*), 43
reset() (*device_interfaces.Compressor method*), 49
reset() (*device_interfaces.ForePump method*), 52
reset() (*device_interfaces.Gauge method*), 48
reset() (*device_interfaces.Heatswitch method*), 36
reset() (*device_interfaces.Magnet method*), 42
reset() (*device_interfaces.PressureRegulator method*), 37
reset() (*device_interfaces.Thermometer method*), 39
reset() (*device_interfaces.TurboPump method*), 51
reset() (*device_interfaces.Valve method*), 45
reset() (*device_interfaces.WarmupHeater method*), 47
reset() (*device_interfaces.Winch method*), 46
restart_service() (*api_client.APIManager method*), 8
restart_service() (*controller_interfaces.CryostatControl method*), 12
return_pressure (*device_interfaces.Compressor property*), 49
return_to_idle() (*controller_interfaces.CryostatControl method*), 12
return_to_idle() (*controller_interfaces.SampleControl method*), 15
return_to_idle() (*controller_interfaces.TemperatureControl method*), 18
rollback_necessary (*controller_interfaces.SampleControl property*), 15
runstatus (*controller_interfaces.CryostatControl property*), 12
runstatus (*device_interfaces.Compressor property*), 49

S

sample_loaded (*controller_interfaces.SampleControl property*), 15
SampleControl (*class in controller_interfaces*), 13
sections (*controller_interfaces.TemperatureControl property*), 18
sensorunits (*device_interfaces.Thermometer property*), 39
sensorunits_buffer (*device_interfaces.Thermometer property*), 39
sensorunits_median (*device_interfaces.Thermometer property*), 39
sensorunits_rate (*device_interfaces.Thermometer property*), 39
sensorunits_stabilitywindow (*device_interfaces.Thermometer property*), 39
sensorunits_stablebuffer (*device_interfaces.Thermometer property*), 39
sensorunits_timestamp (*device_interfaces.Thermometer property*), 39
sensorunits_transientbuffer (*device_interfaces.Thermometer property*), 39
sensorunits_transientwindow (*device_interfaces.Thermometer property*), 39
sequential (*controller_interfaces.TemperatureControl property*), 18
set() (*api_client.APIManager method*), 8
set() (*api_client.KiutraClient method*), 6
set() (*controller_interfaces.ADRControl method*), 23
set() (*controller_interfaces.Control method*), 33
set() (*controller_interfaces.CryostatControl method*), 12
set() (*controller_interfaces.HeaterControl method*), 27
set() (*controller_interfaces.MagnetControl method*), 30
set() (*controller_interfaces.SampleControl method*), 15
set() (*controller_interfaces.TemperatureControl method*), 18
set() (*device_interfaces.Baffles method*), 43
set() (*device_interfaces.Compressor method*), 49
set() (*device_interfaces.ForePump method*), 52

set() (*device_interfaces.Gauge method*), 48
set() (*device_interfaces.Heatswitch method*), 36
set() (*device_interfaces.Magnet method*), 42
set() (*device_interfaces.PressureRegulator method*), 37
set() (*device_interfaces.Thermometer method*), 40
set() (*device_interfaces.TurboPump method*), 51
set() (*device_interfaces.Valve method*), 45
set() (*device_interfaces.WarmupHeater method*), 47
set() (*device_interfaces.Winch method*), 46
setpoint (*controller_interfaces.ADRControl property*), 23
setpoint (*controller_interfaces.HeaterControl property*), 27
setpoint (*controller_interfaces.MagnetControl property*), 30
setpoint (*controller_interfaces.TemperatureControl property*), 19
setpoint (*device_interfaces.Baffles property*), 43
setpoint (*device_interfaces.Heatswitch property*), 36
setpoint (*device_interfaces.WarmupHeater property*), 47
setpoint (*device_interfaces.Winch property*), 46
short_purge_airlock() (*controller_interfaces.SampleControl method*), 15
stable (*controller_interfaces.ADRControl property*), 23
stable (*controller_interfaces.HeaterControl property*), 27
stable (*controller_interfaces.MagnetControl property*), 30
stable (*controller_interfaces.TemperatureControl property*), 19
start() (*controller_interfaces.ADRControl method*), 23
start() (*controller_interfaces.Control method*), 33
start() (*controller_interfaces.CryostatControl method*), 12
start() (*controller_interfaces.HeaterControl method*), 27
start() (*controller_interfaces.MagnetControl method*), 30
start() (*controller_interfaces.SampleControl method*), 15
start() (*controller_interfaces.TemperatureControl method*), 19
start() (*device_interfaces.ForePump method*), 52
start_adr() (*controller_interfaces.ADRControl method*), 23
start_cadr() (*controller_interfaces.ADRControl method*), 23
start_cadr() (*controller_interfaces.TemperatureControl method*), 19
start_control() (*controller_interfaces.TemperatureControl method*), 19
start_cryocooler() (*controller_interfaces.CryostatControl method*), 12
start_heater_control() (*controller_interfaces.TemperatureControl method*), 19
start_proposed_mode() (*controller_interfaces.TemperatureControl method*), 19
start_sequence() (*controller_interfaces.TemperatureControl method*), 20
start_serial_adr() (*controller_interfaces.ADRControl method*), 24
start_single_adr() (*controller_interfaces.ADRControl method*), 24
start_single_adr() (*controller_interfaces.TemperatureControl method*), 20
state (*controller_interfaces.ADRControl property*), 24
state (*controller_interfaces.HeaterControl property*), 27
state (*controller_interfaces.MagnetControl property*), 31
state (*controller_interfaces.TemperatureControl property*), 20
status (*api_client.APIManager property*), 8
status (*controller_interfaces.ADRControl property*), 24
status (*controller_interfaces.Control property*), 33
status (*controller_interfaces.CryostatControl property*), 12
status (*controller_interfaces.HeaterControl property*), 27
status (*controller_interfaces.MagnetControl property*), 31
status (*controller_interfaces.SampleControl property*), 15
status (*controller_interfaces.TemperatureControl property*), 20
status (*device_interfaces.Baffles property*), 44
status (*device_interfaces.Compressor property*), 50
status (*device_interfaces.ForePump property*), 52
status (*device_interfaces.Gauge property*), 48
status (*device_interfaces.Heatswitch property*), 36
status (*device_interfaces.Magnet property*), 42
status (*device_interfaces.PressureRegulator property*), 37
status (*device_interfaces.Thermometer property*), 40
status (*device_interfaces.TurboPump property*), 51
status (*device_interfaces.Valve property*), 45
status (*device_interfaces.WarmupHeater property*), 47
status (*device_interfaces.Winch property*), 46
step (*controller_interfaces.ADRControl property*), 24
step (*controller_interfaces.Control property*), 33
step (*controller_interfaces.CryostatControl property*), 12
step (*controller_interfaces.HeaterControl property*), 27
step (*controller_interfaces.MagnetControl property*), 31
step (*controller_interfaces.SampleControl property*), 15
step (*controller_interfaces.TemperatureControl property*), 20
stop() (*controller_interfaces.ADRControl method*), 24
stop() (*controller_interfaces.Control method*), 33
stop() (*controller_interfaces.CryostatControl method*), 12
stop() (*controller_interfaces.HeaterControl method*), 27
stop() (*controller_interfaces.MagnetControl method*), 31
stop() (*controller_interfaces.SampleControl method*), 15
stop() (*controller_interfaces.TemperatureControl method*), 20
stop() (*device_interfaces.Baffles method*), 44
stop() (*device_interfaces.ForePump method*), 52
stop() (*device_interfaces.TurboPump method*), 51
stop() (*device_interfaces.WarmupHeater method*), 47
stop() (*device_interfaces.Winch method*), 46
stop_cryocooler() (*controller_interfaces.CryostatControl method*), 12
stop_heater() (*controller_interfaces.CryostatControl method*), 12
stop_pumps() (*controller_interfaces.CryostatControl method*), 12
successful_termination (*controller_interfaces.ADRControl property*), 24

successful_termination (*controller_interfaces.Control property*), 33
successful_termination (*controller_interfaces.CryostatControl property*), 12
successful_termination (*controller_interfaces.HeaterControl property*), 27
successful_termination (*controller_interfaces.MagnetControl property*), 31
successful_termination (*controller_interfaces.SampleControl property*), 15
successful_termination (*controller_interfaces.TemperatureControl property*), 20

T

temperature (*device_interfaces.TurboPump property*), 51
temperature_rate (*controller_interfaces.CryostatControl property*), 13
TemperatureControl (*class in controller_interfaces*), 16
tesla (*controller_interfaces.MagnetControl property*), 31
tesla (*device_interfaces.Magnet property*), 42
tesla_buffer (*controller_interfaces.MagnetControl property*), 31
tesla_buffer (*device_interfaces.Magnet property*), 42
tesla_median (*controller_interfaces.MagnetControl property*), 31
tesla_median (*device_interfaces.Magnet property*), 42
tesla_rate (*controller_interfaces.MagnetControl property*), 31
tesla_rate (*device_interfaces.Magnet property*), 42
tesla_stabilitywindow (*controller_interfaces.MagnetControl property*), 31
tesla_stabilitywindow (*device_interfaces.Magnet property*), 42
tesla_stablebuffer (*controller_interfaces.MagnetControl property*), 31
tesla_stablebuffer (*device_interfaces.Magnet property*), 42
tesla_timestamp (*controller_interfaces.MagnetControl property*), 31
tesla_timestamp (*device_interfaces.Magnet property*), 42
tesla_transientbuffer (*controller_interfaces.MagnetControl property*), 31
tesla_transientbuffer (*device_interfaces.Magnet property*), 42
tesla_transientwindow (*controller_interfaces.MagnetControl property*), 31
tesla_transientwindow (*device_interfaces.Magnet property*), 42
Thermometer (*class in device_interfaces*), 37
time() (*api_client.APIManager method*), 8
time() (*controller_interfaces.ADRControl method*), 24
time() (*controller_interfaces.Control method*), 33
time() (*controller_interfaces.CryostatControl method*), 13
time() (*controller_interfaces.HeaterControl method*), 27
time() (*controller_interfaces.MagnetControl method*), 31
time() (*controller_interfaces.SampleControl method*), 15
time() (*controller_interfaces.TemperatureControl method*), 20
time() (*device_interfaces.Baffles method*), 44
time() (*device_interfaces.Compressor method*), 50
time() (*device_interfaces.ForePump method*), 53
time() (*device_interfaces.Gauge method*), 48
time() (*device_interfaces.Heatswitch method*), 36
time() (*device_interfaces.Magnet method*), 42
time() (*device_interfaces.PressureRegulator method*), 37
time() (*device_interfaces.Thermometer method*), 40
time() (*device_interfaces.TurboPump method*), 51
time() (*device_interfaces.Valve method*), 45
time() (*device_interfaces.WarmupHeater method*), 47
time() (*device_interfaces.Winch method*), 46
transfer_sample() (*controller_interfaces.SampleControl method*), 16
trouble_shoot() (*controller_interfaces.CryostatControl method*), 13
TurboPump (*class in device_interfaces*), 50

U

unload_sample() (*controller_interfaces.SampleControl method*), 16
unlock() (*api_client.APIManager method*), 8
unlock_with_timeout() (*api_client.APIManager method*), 8

V

Valve (*class in device_interfaces*), 44
vent() (*controller_interfaces.CryostatControl method*), 13
vent_airlock() (*controller_interfaces.CryostatControl method*), 13
vent_airlock() (*controller_interfaces.SampleControl method*), 16
vent_dewar() (*controller_interfaces.CryostatControl method*), 13
voltage (*controller_interfaces.MagnetControl property*), 31
voltage (*device_interfaces.Compressor property*), 50
voltage (*device_interfaces.Gauge property*), 48
voltage (*device_interfaces.Magnet property*), 42
voltage (*device_interfaces.PressureRegulator property*), 37
voltage (*device_interfaces.TurboPump property*), 51
voltage (*device_interfaces.WarmupHeater property*), 47
voltage_rate (*device_interfaces.Magnet property*), 43

W

wait_done() (*controller_interfaces.ADRControl method*), 24
wait_done() (*controller_interfaces.HeaterControl method*), 27
wait_done() (*controller_interfaces.MagnetControl method*), 31
wait_done() (*controller_interfaces.TemperatureControl method*), 20
wait_stable() (*controller_interfaces.ADRControl method*), 24
wait_stable() (*controller_interfaces.HeaterControl method*), 28

wait_stable() (*controller_interfaces.MagnetControl method*), 31
wait_stable() (*controller_interfaces.TemperatureControl method*), 21
warmup() (*controller_interfaces.CryostatControl method*), 13
WarmupHeater (*class in device_interfaces*), 46
waterflow_error (*device_interfaces.Compressor property*), 50
watertemp_error (*device_interfaces.Compressor property*), 50
Winch (*class in device_interfaces*), 45